주문·재고 운영 관리

주문·재고의 정확한 처리부터 로그인과 배포까지

Admin Portfolio

주문 한 건이 저장된 뒤에도
재고·이벤트·통계가 이어지도록 설계했습니다.

프로젝트 목적

단순 CRUD를 출발점으로 동시 재고 변경, 외부 메시지 전송 실패, 중복 이벤트와 인증·권한 문제를 다뤘습니다. React 화면으로 업무 흐름을 확인하고 AWS에 실제 배포했습니다.

구현 범위

Spring Boot API · MySQL 스키마와 조회 · Redis 세션/캐시/요청 제한 · Kafka/Outbox · Keycloak OIDC · React 관리 화면 · Jenkins와 AWS 배포

읽는 순서

구성도 → 주문·재고 처리 → 전송 재시도·중복 반영 방지 → 인증 → 배포·화면

실행 서비스 https://13-124-81-142.sslip.io

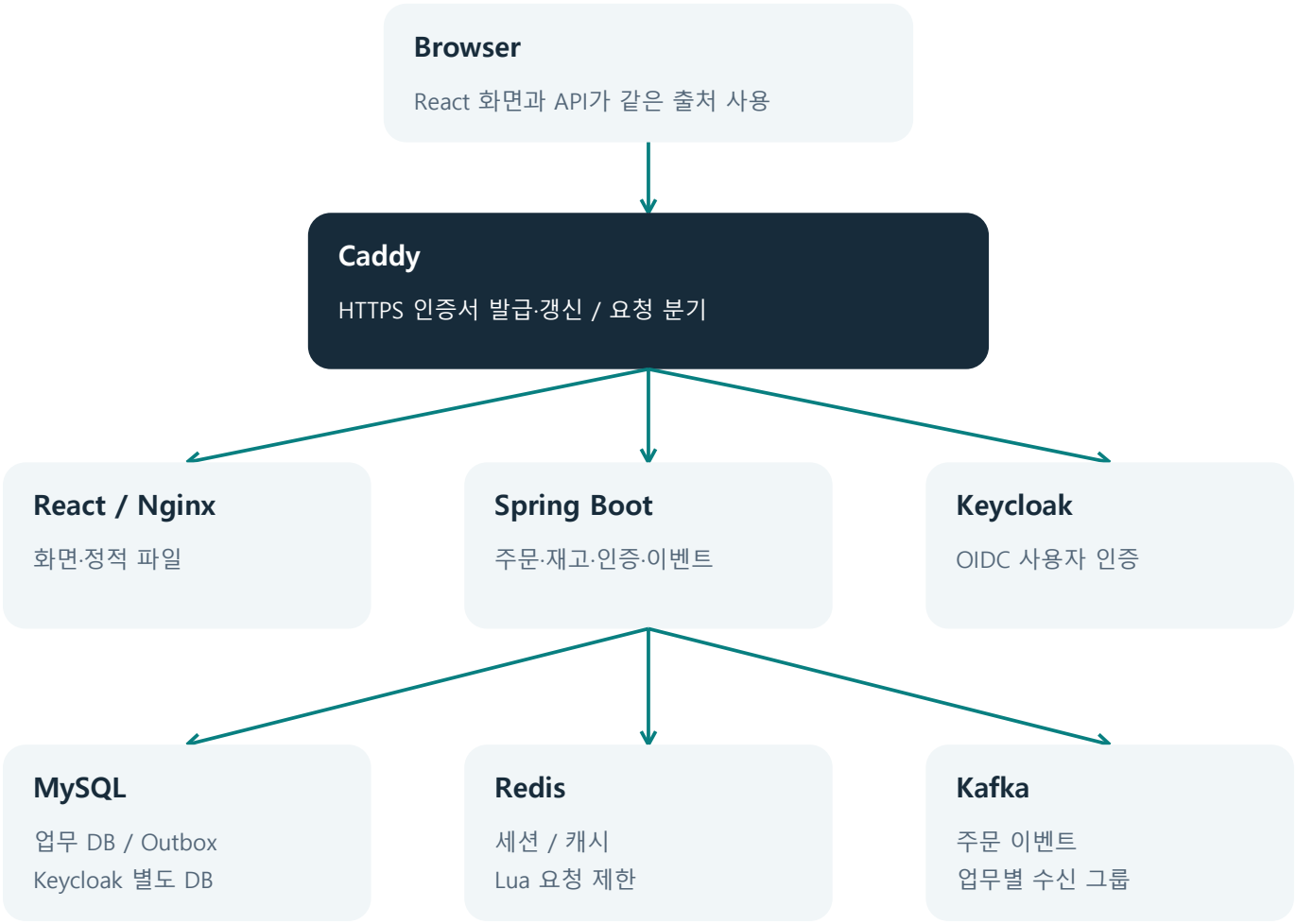
Backend github.com/whdcks0101-beep/admin-api

Frontend github.com/whdcks0101-beep/admin-frontend

공개 체험 관리자와 일반회원 로그인 정보는 마지막 페이지에서 확인하세요.

HTTPS 주소 하나로 연결

AWS Lightsail / Ubuntu / Docker Compose



Spring Boot는 Keycloak과 토큰을 교환하고, Kafka 이벤트를 소비해 MySQL 통계를 갱신합니다. 위 화살표는 주요 연결을 요약합니다.

클라우드 실행 경계

백엔드 한 인스턴스와 인프라를 단일 Lightsail 서버에서 실행합니다. Caddy만 공개 HTTP/HTTPS 포트를 사용하고 DB·Redis·Kafka는 내부 통신으로 연결합니다.

로컬 Kubernetes 경험

별도 Docker Desktop 환경에서 백엔드 2개 Pod, Redis 세션 공유, Pod 재생성, 상태 점검, 롤링 업데이트와 롤백을 실습했습니다.

재고 충돌과 부분 저장 방지

문제 → 트랜잭션 경계 → 충돌 처리 → 검증

문제

두 요청이 같은 수량을 읽고 각각 저장하면 한 변경이 사라질 수 있습니다. 다품목 주문 중 일부 재고만 부족한 경우에도 주문과 차감 내역이 함께 취소되어야 합니다.

선택한 구현

Inventory에 JPA @Version을 적용해 오래된 버전의 변경을 거부합니다. OrderService.createOrder의 쓰기 트랜잭션에 주문·주문 항목·재고 차감·Outbox 저장을 함께 포함했습니다.

하나의 DB 트랜잭션

주문 저장 → 재고 차감 → 주문 항목 저장 → Outbox 저장
모두 커밋하거나 모두 롤백

사용자에게 보이는 동작

재고 부족은 주문 실패로 안내합니다. 동시 재고 수정으로 충돌하면 HTTP 409로 안내하고, 사용자가 최신 상태를 조회해 다시 판단하도록 합니다.

조회와 저장의 역할

QueryDSL로 주문 번호·기간 조건을 조합하고 페이징을 제공합니다. 상품 정보가 필요할 때는 함께 조회합니다(fetch join). 작업 할당과 중복 저장 방지에는 MySQL SQL을 직접 사용했습니다.

검증 근거

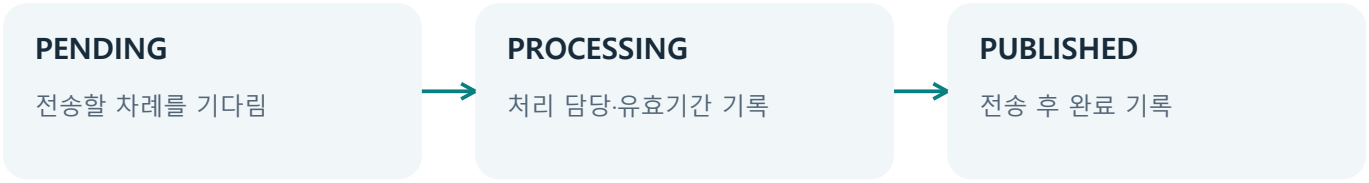
InventoryOptimisticLockTest, OrderServiceRollbackTest, OrderOutboxTransactionTest로 충돌·전체 롤백·동시 저장을 검증합니다. AWS 화면에서 주문 생성과 재고 차감도 확인했습니다.

같은 이벤트가 와도 통계는 한 번만

이벤트를 DB에 보관하고 재시도 / 통계 중복 반영 방지(멱등 처리)

전송에 실패한 이벤트를 다시 보내려면

주문과 전송할 이벤트를 DB에 함께 저장합니다(Outbox). 전송 작업자(Relay)가 저장된 이벤트를 Kafka로 보냅니다. 전송에 실패해도 DB에 남은 이벤트를 다시 보낼 수 있습니다.



여러 서버가 작업을 나눠 처리할 때

다른 작업자가 잠근 행은 건너뜁니다(FOR UPDATE SKIP LOCKED). 처리 담당을 기록한 뒤 DB 잠금을 풀고 전송합니다. 결과를 저장할 때는 처리 토큰(claim token)과 유효기간을 DB 시간으로 확인합니다.

중복 전송을 전제로 통계 보호

전송 성공 후 완료 기록에 실패하면 이벤트가 다시 전달될 수 있습니다. order_statistics의 order_id 고유 키와 ON DUPLICATE KEY UPDATE로 동일 주문의 통계 중복 저장을 방지합니다.

처리에 실패한 메시지는 따로 보관

다시 시도해도 처리하지 못한 메시지는 별도 보관 장소(DLT)로 보냅니다. JSON을 읽지 못하면 원문도 보존합니다. 로그와 통계는 별도 수신 그룹(Consumer Group)에서 처리합니다.

검증과 해석

같은 주문 이벤트를 다시 처리해도 통계 건수가 늘지 않는 것을 확인했습니다. 이벤트는 여러 번 전달될 수 있지만, 같은 주문의 통계는 한 번만 저장합니다.

인증과 서비스 권한을 분리

Keycloak OIDC / Spring Security / Redis

SSO 연결

Keycloak은 사용자 본인 인증을 담당합니다. Spring Security가 issuer + subject로 연결된 서비스 회원을 찾고, 우리 DB의 회원 구분으로 권한을 부여합니다. 이메일 일치만으로 관리자 계정을 자동 연결하지 않습니다.

Keycloak 인증 → 회원 연결 조회 → 서비스 권한 적용

사전 등록 방식 / 미연결 계정 거부 / SSO 로그아웃 확인

세션과 접근 제어

Spring Session Redis로 인스턴스 간 세션을 공유합니다. 변경 요청에는 CSRF 보호를 적용하고 인증 실패와 권한 부족은 API에서 401/403으로 구분합니다. AWS 세션 쿠키에 Secure·HttpOnly를 적용했습니다.

상품 수정 후 예전 캐시 삭제

상품 조회 캐시를 사용하고 수정 시 캐시 삭제 요청을 Outbox에 저장합니다. 편집용 조회는 DB를 직접 읽습니다. 재고처럼 최신 상태 판단이 필요한 업무와 캐시 사용 범위를 구분했습니다.

요청 횟수 제한

Redis Lua로 요청 횟수 증가와 만료 설정 사이에 다른 요청이 끼어들지 않게 합니다. 여러 서버가 같은 제한을 공유하며 초과 시 429로 안내합니다. 요청 횟수 제한만으로 중복 주문 생성을 막지는 않습니다.

확인한 결과

AWS에서 일반 로그인, SSO 로그인, 관리자 메뉴 접근, SSO 로그아웃을 확인했습니다. 로컬에서는 Redis 세션 공유와 요청 제한, 예전 캐시 삭제를 검증했습니다.

버튼 한 번으로 AWS 배포

Jenkins는 개발 PC에서, 서비스는 AWS에서 실행

01 GitHub checkout

백엔드·프론트 저장소 분리

02 테스트와 이미지 빌드

백엔드 테스트 / 프론트 린트·테스트

03 SSH 이미지 전송

서버 공개키 검증 / Jenkins Secret file

04 컨테이너 교체·상태 확인

배포 잠금 / 성공한 이미지 태그만 기록

추적 가능한 배포

aws-빌드번호-Git커밋 태그를 사용합니다. frontend와 backend를 각각 배포하고 서버 잠금으로 변경을 직렬화합니다. 단일 컨테이너 교체 중에는 잠시 연결이 끊길 수 있으며 실패 시 자동 롤백하지 않습니다.

테스트 실행 기록

2026-10-26 로컬 기본 테스트: 88개 중 69개 통과, 19개 조건부 건너뜀, 실패 0개. MySQL·실제 Redis 통합 테스트는 별도 활성화 대상입니다. 프론트 빌드에는 린트와 테스트 단계가 포함됩니다.

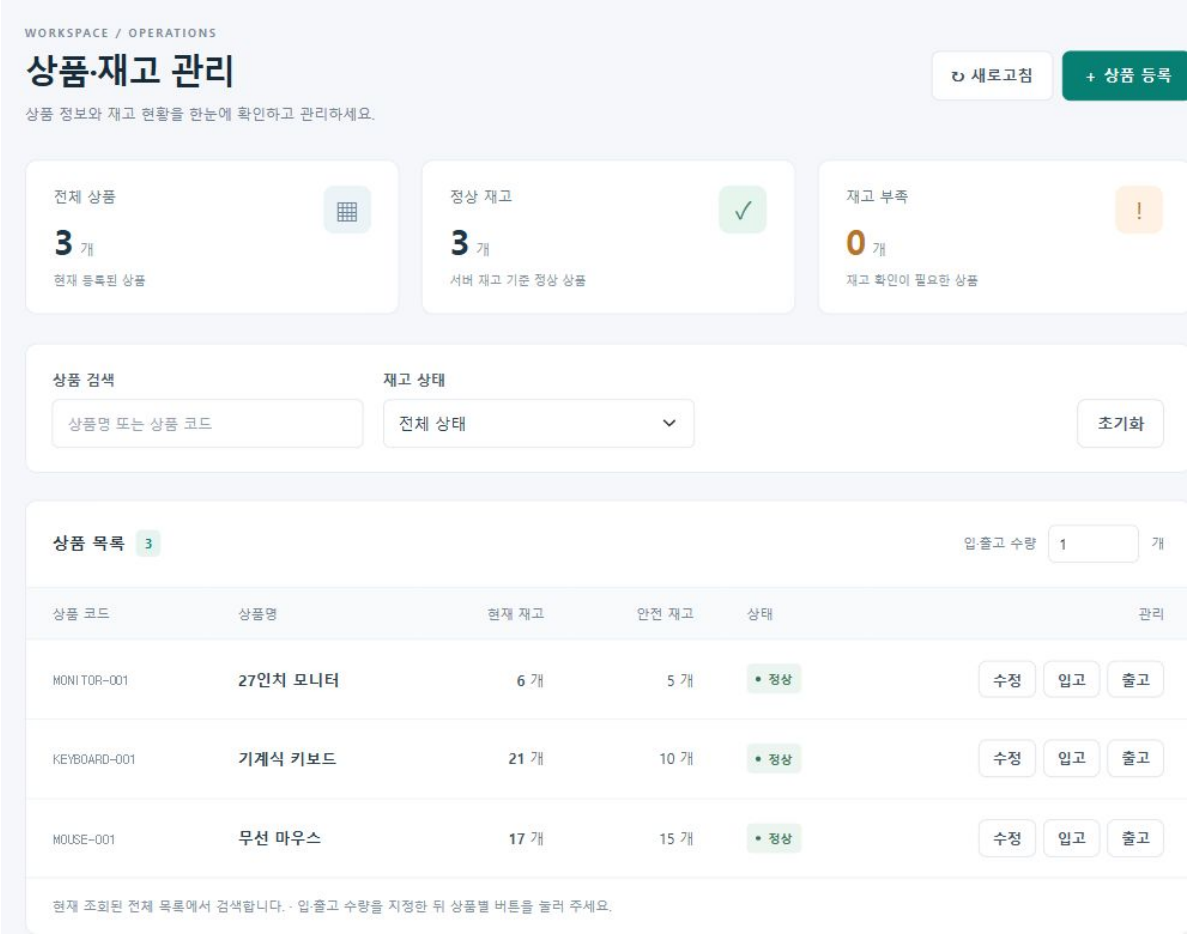
클라우드 사용자 검증

백엔드·프론트 Jenkins 작업 성공 후 실제 로그인·목록 조회를 확인했습니다. 주문 생성, 재고 차감, Kafka를 통한 날짜별 통계 반영과 SSO 로그아웃도 확인했습니다.

상품·재고에서 주문 상세까지

2026-10-26 AWS 서비스 캡처 / 반응형 화면 / 샘플 데이터

상품·재고 관리

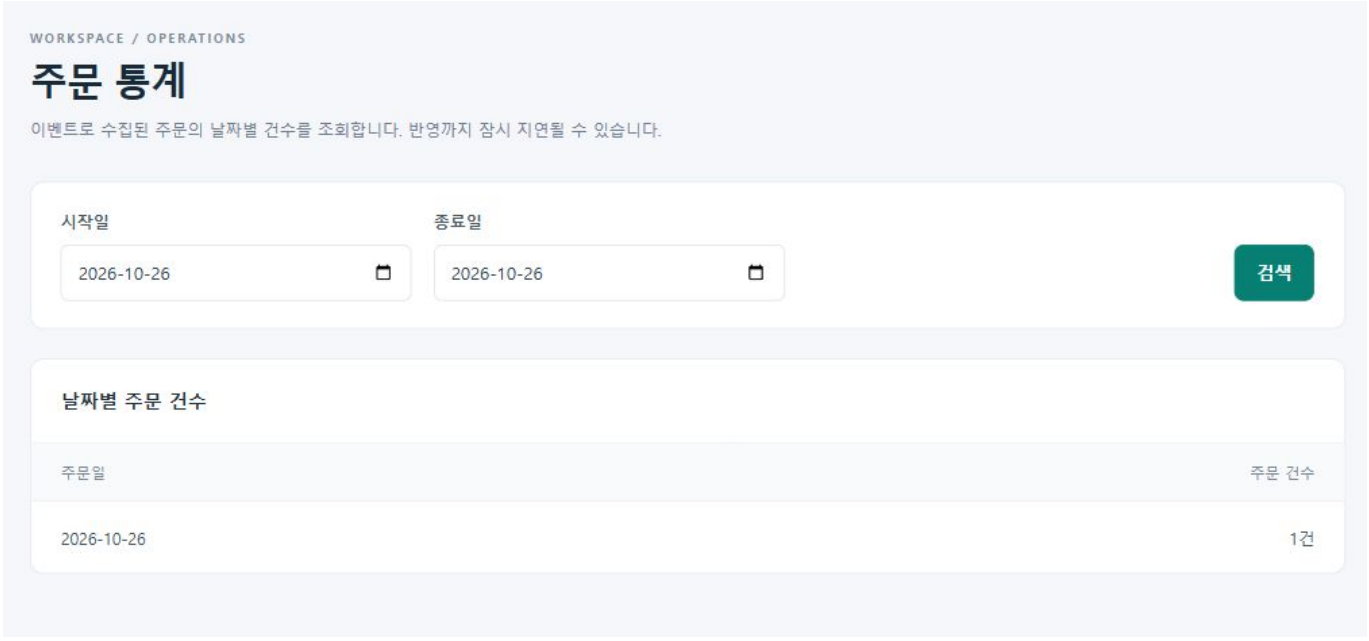


주문 상세



이벤트가 통계로 이어지는 결과

주문 생성 후 비동기로 반영된 일별 주문 건수



확인한 업무 흐름

주문·재고·이벤트 함께 저장 → 전송 작업자 → Kafka
→ 통계 처리 → 일별 주문 건수 조회

화면에서 확인한 결과

2026-10-26 주문 1건이 통계에 반영된 실제 조회 결과입니다. 비동기 처리이므로 주문 저장 직후에는 잠시 지연될 수 있습니다.

구현 사례와 소스: GitHub backend README

실행 서비스: https://13-124-81-142.sslip.io

직접 로그인하고 권한 비교하기

운영 관리자와 분리한 공개 체험 계정

서비스 열기 https://13-124-81-142.sslip.io

체험 관리자 | 이메일 로그인 / SSO

아이디: demo-admin@example.com
비밀번호: PortfolioDemo2026!

일반회원 | 이메일 로그인

아이디: user1@gmail.com
비밀번호: normalUser1234

공통으로 체험할 기능

상품·재고 조회, 주문 생성·상세 조회, 날짜별 주문 통계를 확인할 수 있습니다.

체험 관리자에게 추가로 허용

상품 등록·수정과 입출고를 허용합니다. 일반회원은 상품 변경 버튼이 표시되지 않으며 API 직접 요청도 거부됩니다.

계정 관리 권한 분리

회원 관리와 권한 변경은 실제 운영 관리자만 허용합니다. 체험 관리자와 일반회원은 회원 API 직접 호출도 HTTP 403으로 차단됩니다.

상품·재고·주문은 방문자들이 함께 쓰는 샘플 데이터입니다. 개인정보는 입력하지 마세요. 실제 운영 관리자 비밀번호는 공개하지 않습니다.